COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S1
DATE PERFORMED:	August 22, 2026	**DATE SUBMITTED:**	August 22, 2026
NAME:	Nichollo Dave A. Galvez	**INSTRUCTOR:**	Engr Jimlord Quejado

ACTIVITY NO. ___ [ACTIVITY NAME]

1. PROCEDURE OUTPUT

ILOA
Main.cpp

```cpp
#include <iostream>
#include <queue>

void display(std::queue<char> r);


int main(){

  std::queue<char> q;
  q.push('c');
  q.push('p');
  q.push('e');
  q.push('0');
  q.push('1');
  q.push('0');


  //DISPLAY
  display(q);

//EMPTY

std::cout << "The queue is empty: " << q.empty();
std::cout<< "\n";

//SIZE

std::cout << "Queue size: " << q.size();
std::cout<< "\n";

```

```
31  //FRONT
32
33  std::cout << "The front is: " << q.front();
34  std::cout<< "\n";
35
36  //BACK
37
38  std::cout << "The back is: " << q.back();
39  std::cout<< "\n";
40
41  //POP
42
43  q.pop();
44
45  //DISPLAY
46
47  display(q);
48
49  //PUSH ANOTHER ELEMENT
50
51  q.push('N');
52  q.push('I');
53
54  //DISPLAY
55
56  display(q);
57
58  }
59
60  void display(std::queue<char> r){
61    //create a copy of the queue
62    std::queue<char> c = r;
63
64    while(!c.empty()){
65      std::cout<< " " << c.front();
66      c.pop();
67
68    }
69    std::cout << "\n";
70
71  }
```

Output:

```
 c p e 0 1 0
The queue is empty: 0
Queue size: 6
The front is: c
The back is: 0
 p e 0 1 0
 p e 0 1 0 N I
```

ILOB_1

Main.cpp

```cpp
#include <iostream>
#include "queue_ll.h"

int main() {

    qNode<char> *front =nullptr;
    qNode<char> *back = nullptr;

    std::cout << "Current Queue: ";
    enqueue(&front, &back, 'N');
    enqueue(&front, &back, 'I');
    enqueue(&front, &back, 'C');
    enqueue(&front, &back, 'H');
    enqueue(&front, &back, 'O');
    display(front);
    dequeue(&front, &back);
    std::cout << front->data << " " << back->data << std::endl;

    return 0;
}
```

queue_ll.h

```cpp
#ifndef QUEUELL_H
#define QUEUELL_H

template <typename T>
class qNode {
public:
    T data;
    qNode* next;
};


//creating a new node
template <typename T>
qNode<T>* new_node(T newData) {
    qNode<T>* newNode = new qNode<T>;
    newNode->data = newData;
    newNode->next = nullptr;
    return newNode;
}


template <typename T>
void enqueue(qNode<T> **frontPtr, qNode<T> **backPtr, T newData) {
    //create a new node
    qNode<T> *newNode = new_node<T>(newData);


    //inserting to an empty queue
    if ((*frontPtr) == nullptr && (*backPtr) == nullptr) {
        (*frontPtr) = newNode;
        (*backPtr) = newNode;
    }


    //inserting an item into a non empty queue
    (*backPtr)->next = newNode; //point the backPtr next to the newNode
    (*backPtr) = newNode;
}


template <typename T>
void dequeue(qNode<T> **frontPtr, qNode<T> **backPtr) {
    //create a temporary node to store the node to be deleted
    qNode<T> *deleteNode = nullptr;
    deleteNode = (*frontPtr);
```

```
48      //check if the queue is only 1 node
49      if ((*frontPtr)->next == nullptr && (*backPtr)->next == nullptr) {
50          (*frontPtr) = nullptr;
51          (*backPtr) = nullptr;
52          delete deleteNode;
53          return;
54      }
55
56
57      //deleting of the node
58      (*frontPtr) = deleteNode->next;
59      delete deleteNode;
60  }
61
62
63  //display all elements in the list
64  template <typename T>
65  void display(qNode<T>* front) {
66      //create a temporary node
67      qNode<T> *displayNode = front;
68
69
70
71
72      while (displayNode != nullptr) {
73          std::cout << displayNode->data;
74          if (displayNode->next != nullptr)
75              std::cout << " -> ";
76
77
78          displayNode = displayNode ->next;
79      }
80
81
82      std::cout << std::endl;
83  }
84
85
86  //return the front variable
87
88
89  #endif
```

Output:

```
Current Queue: N->I->C->H->O->NULL
N
```

ILOB_2

Main.cpp

```cpp
#include <iostream>
#include "queueArr.h"

int main(){

    //create an objecy

//enqueue
    queueArr<int> q(6);
    q.Enqueue(1);
    std::cout << "q_front: " << q.Front() << ", q_back: " << q.Back() << ", q_size: " << q.size() << std::endl;

    q.Enqueue(2);
    std::cout << "q_front: " << q.Front() << ", q_back: " << q.Back() << ", q_size: " << q.size() << std::endl;

    q.Enqueue(3);
    std::cout << "q_front: " << q.Front() << ", q_back: " << q.Back() << ", q_size: " << q.size() << std::endl;

    q.Enqueue(4);
    std::cout << "q_front: " << q.Front() << ", q_back: " << q.Back() << ", q_size: " << q.size() << std::endl;

    q.Enqueue(5);
    std::cout << "q_front: " << q.Front() << ", q_back: " << q.Back() << ", q_size: " << q.size() << std::endl;

    q.Enqueue(6);
    std::cout << "q_front: " << q.Front() << ", q_back: " << q.Back() << ", q_size: " << q.size() << std::endl;

// dequeue
    q.Dequeue();
    std::cout << q.Front() << ", q_back: " << q.Back() << ", q_size: " << q.size() << std::endl;

    q.Dequeue();
    std::cout << q.Front() << ", q_back: " << q.Back() << ", q_size: " << q.size() << std::endl;

    q.Dequeue();
    std::cout << q.Front() << ", q_back: " << q.Back() << ", q_size: " << q.size() << std::endl;

    q.Dequeue();
    std::cout << q.Front() << ", q_back: " << q.Back() << ", q_size: " << q.size() << std::endl;

    q.Dequeue();
    std::cout << q.Front() << ", q_back: " << q.Back() << ", q_size: " << q.size() << std::endl;

    q.Dequeue();
    std::cout << q.Front() << ", q_back: " << q.Back() << ", q_size: " << q.size() << std::endl;

    q.Dequeue();
    std::cout << q.Front() << ", q_back: " << q.Back() << ", q_size: " << q.size() << std::endl;
    return 0;

// copy constructor
    queueArr<int> q2(q);
    std::cout << "q2_front: " << q2.Front() << ", q2_back: " << q2.Back() << ", q2_size: " << q2.size() << std::endl;

// assignment operator
    queueArr<int> q3(6);
    q3 = q;
    std::cout << "q3_front: " << q3.Front() << ", q3_back: " << q3.Back() << ", q3_size: " << q3.size() << std::endl;

}
```

queueArr.h

```cpp
#ifndef QUEUEARR_H
#define QUEUEARR_H

#include <iostream>
template <typename T>
class queueArr{
    private:
        // create apointer array:
        T* q_array;
        size_t q_capacity;
        size_t q_size;
        int q_front;
        int q_back;

    public:
        //prototypes to use for queueArr:
        queueArr(size_t capacity);
        bool empty();
        bool full();

        size_t size();
        void Clear();

        T Front();
        T Back();

        void Enqueue(T newData);
        void Dequeue();

        ~queueArr();

        //tasks
        //copy constructor
        queueArr(const queueArr& other);

        //assignment operator
        queueArr& operator=(const queueArr& other);
};

// constructor
template <typename T>
queueArr<T>::queueArr(size_t capacity){

    //capacity of the array
    q_capacity = capacity;

```

```
47
48      //current size of the array
49      q_size = 0;
50
51
52      //initial front and back of the index
53      q_front = 0;
54      q_back = -1;
55
56
57      // newly allocated array
58      q_array = new T[q_capacity];
59      std::cout << "Queue created with capacity: " << q_capacity << std::endl;
60  }
61
62
63  //destructor
64  template <typename T>
65  queueArr<T>::~queueArr(){
66
67      delete [] q_array;
68      std::cout << "Queue destroyed." << std::endl;
69  }
70
71
72
73
74  //empty
75  template <typename T>
76  bool queueArr<T>::empty(){
77      return q_size == 0;
78  }
79
80
81  //full
82  template <typename T>
83  bool queueArr<T>::full(){
84      return q_size == q_capacity;
85  }
86
87
```

```cpp
//clear
template <typename T>
void queueArr<T>::Clear(){
    q_size = 0;
    q_front = 0;
    q_back = -1;
}


//return the element in the front of the queue
template <typename T>
T queueArr<T>::Front(){
    //check and display if the queue is empty
    if (empty()){
        std::cout << "Queue is empty. \n" << std::endl;
        return T{}; // return default value of T
    }
    //return the front element
    return q_array[q_front];
}


// back
template <typename T>
T queueArr<T>::Back(){
    //check and display if the queue is empty
    if (empty()){
        std::cout << "Queue is empty. \n" << std::endl;
        return T{}; // return default value of T
    }
    // if not then return the back element
    return q_array[q_back];
}


template <typename T>
size_t queueArr<T>::size(){
    return q_size;
}

```

```cpp
template <typename T>
void queueArr<T>::Enqueue(T newData){
    // check if the queue is full
    if (full()){
        std::cout << "Queue is full." << std::endl;
        return;
    }


    // compute for the q_back to move it circularly
    q_back = (q_back + 1) % q_capacity;


    //assign the new value to the back of the queue array
    q_array[q_back] = newData;
    //increment the q_size
    q_size++;
}


//dequeue
template <typename T>
void queueArr<T>::Dequeue(){
    // check if the queue is empty
    if (empty()){
        std::cout << "Queue is empty." << std::endl;
        return;
    }


    // compute for q_front to move it circularly
    q_front = (q_front + 1) % q_capacity;


    //decrement the size q_size
    q_size--;
}

// copy constructor
template <typename T>
queueArr<T>::queueArr(const queueArr& other) {
    q_capacity = other.q_capacity;
    q_size = other.q_size;
    q_front = other.q_front;
```

```
173         q_back = other.q_back;
174
175
176         // Allocate new memory for the array
177         q_array = new T[q_capacity];
178
179
180         // Copy the elements from the other queue
181         for (size_t i = 0; i < q_size; ++i) {
182             size_t index = (q_front + i) % q_capacity;
183             q_array[index] = other.q_array[index];
184         }
185     }
186
187
188     // copy assignment operator
189     template <typename T>
190     queueArr<T>& queueArr<T>::operator=(const queueArr& other) {
191         if (this != &other) {
192             // Clean up existing memory
193             delete [] q_array;
194
195
196             // Copy properties from the other queue
197             q_capacity = other.q_capacity;
198             q_size = other.q_size;
199             q_front = other.q_front;
200             q_back = other.q_back;
201
202
203             // Allocate new memory for the array
204             q_array = new T[q_capacity];
205
206
207             // Copy the elements from the other queue
208             for (size_t i = 0; i < q_size; ++i) {
209                 size_t index = (q_front + i) % q_capacity;
210                 q_array[index] = other.q_array[index];
211             }
212         }
213         return *this;
214     }
215     #endif
216
```

Output:

```
Queue created with capacity: 6
q_front: 1, q_back: 1, q_size: 1
q_front: 1, q_back: 2, q_size: 2
q_front: 1, q_back: 3, q_size: 3
q_front: 1, q_back: 4, q_size: 4
q_front: 1, q_back: 5, q_size: 5
q_front: 1, q_back: 6, q_size: 6
2, q_back: 6, q_size: 5
3, q_back: 6, q_size: 4
4, q_back: 6, q_size: 3
5, q_back: 6, q_size: 2
6, q_back: 6, q_size: 1
Queue is empty.

0, q_back: Queue is empty.

0, q_size: 0
Queue is empty.
Queue is empty.

0, q_back: Queue is empty.

0, q_size: 0
Queue destroyed.
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Array

```cpp
#include <iostream>
#include <string>

// create a class called job
class Job {
    private:
        // Id number for the job
        int id;
        // name of the user who submitted the job
        std::string name;
        // number of pages for the printer
        int pages;
```

```
14      public:
15          // default constructor
16          Job() {
17              id = 0;
18              name = "";
19              pages = 0;
20          }
21
22          // constructor for the job class
23          Job(int i, std::string n, int p) {
24              id = i;
25              name = n;
26              pages = p;
27          }
28
29          // function to print out the job's details
30          void display() {
31              std::cout << "Job ID: " << id << ", User: " << name << ", Pages: " << pages << std::endl;
32          }
33  };
34
35  class Printer {
36      private:
37          const int capacity = 10;
38          Job jobs[10];
39          int front;
40          int rear;
41          int count;
42
43      public:
44          // create a queue to hold the jobs
45          Printer() {
46              front = 0;
47              rear = -1;
48              count = 0;
49          }
50
51          // checks if the queue is full
52          bool isFull() {
53              return count == capacity;
54          }
55
56          // Check if queue is empty
57          bool isEmpty() {
58              return count == 0;
59          }
```

```
    }

    // function to add a job to the queue
    void addJob(Job newJob) {
        if (isFull()) {
            std::cout << "Printer queue is full. Cannot add new job." << std::endl;
            return;
        }
        rear = (rear + 1) % capacity; // this moves the rear index circularly
        jobs[rear] = newJob;
        count++;
        std::cout << "Job added to the printer queue." << std::endl;
    }

    // function to process the job from the queue
    void processJob() {
        std::cout << "\nProcessing job:\n" << std::endl;

        while (!isEmpty()) {
            std::cout << "Now printing....." << std::endl;
            jobs[front].display();
            front = (front + 1) % capacity; // this moves the front index circularly
            count--;
            std::cout << std::endl;
        }
    }

    // shows the waiting line for the jobs awaiting to be processed
    void showQueue() {
        if (isEmpty()) {
            std::cout << "\nPrinter queue is empty." << std::endl;
            return;
        }

        std::cout << "\nJobs in the printer queue:" << std::endl;
        int index = front;
        for (int i = 0; i < count; i++) {
            jobs[index].display();
            index = (index + 1) % capacity; // this moves the index circularly
        }
        std::cout << std::endl;
    }
};

```

```cpp
int main() {
    // creating of objects for the printer and jobs
    Printer printer;

    // Hardcoded job additions
    printer.addJob(Job(1, "Nicho Galvez", 12));
    printer.addJob(Job(2, "Lebron James", 5));
    printer.addJob(Job(3, "Stephen Curry", 20));

    // Show the jobs in the queue
    printer.showQueue();

    // Process the jobs in the queue
    printer.processJob();

    // Show the jobs in the queue after processing
    printer.showQueue();

    return 0;
}
```

```
Job added to the printer queue.
Job added to the printer queue.
Job added to the printer queue.

Jobs in the printer queue:
Job ID: 1, User: Nicho Galvez, Pages: 12
Job ID: 2, User: Lebron James, Pages: 5
Job ID: 3, User: Stephen Curry, Pages: 20


Processing job:

Now printing.....
Job ID: 1, User: Nicho Galvez, Pages: 12

Now printing.....
Job ID: 2, User: Lebron James, Pages: 5

Now printing.....
Job ID: 3, User: Stephen Curry, Pages: 20


Printer queue is empty.
```

Summary of lessons learned

● Analysis of the procedure

- Using the STL queue demonstrated how built-in functions like push(), pop(), and front() simplify code by handling memory automatically. In the linked list approach, I managed dynamic node allocation during enqueue and dequeue operations, ensuring empty and single-node edge cases were handled properly. For the array implementation, I used modulo arithmetic to create a circular buffer, successfully preventing index drift and reusing array space efficiently.

● Analysis of the supplementary activity

- In the printer simulation, I used a circular array (jobs[10]) inside the Printer class to model physical buffer memory. When users like LeBron James and Stephen Curry submit print requests, addJob() enqueues them at the back via modulo arithmetic (rear = (rear + 1) % capacity). processJob() then dequeues tasks from the front, ensuring LeBron's job prints before Stephen's if submitted first. This guarantees strict First-In, First-Out (FIFO) processing with time complexity.

● Concluding statement / Feedback: How well did you think you did in this activity? What are your areas for improvements

- Although I was able to successfully run the code and pass the activity, I still feel unconfident in my grasp of custom queue structures. I struggled to fully track how the front and rear indices move in a circular array, and managing pointers in the linked list implementation felt overwhelming at times. I definitely need more practice writing these data structures from scratch, particularly when it comes to handling dynamic array resizing and properly managing edge cases like queue overflow and underflow.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Quillbot, Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction, Idea generation**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

COMPUTER ENGINEERING

QUEZON CITY

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.